SECURENOVA

AI IDENTITY SECURITY POLICY

Identity Lifecycle | Credential Governance | Incident Response

Table of Contents

Chapter	Title
1	**Identity Lifecycle Management**
1.1	Purpose and scope
1.2	Identity types and ownership
1.3	Provisioning and approval
1.4	Access assignment and review
1.5	Credential and identity rotation
1.6	Suspension and decommissioning
1.7	Audit and evidence requirements
2	**Credential Governance**
2.1	Credential Principles
2.2	Credential rotation schedule
2.3	Prohibition on long-lived agent credentials
2.4	Secrets storage and handling
2.5	Access control, monitoring and emergency rotation
3	**Incident Response Playbook**
3.1	Response process and severity
3.2	Scenario-A – Leaked LLM API key
3.3	Scenario-B – Compromised agent Identity
3.4	Scenario-C – Prompt injection leading to data exfiltration
3.5	Post-incident review and lessons learned

Policy Statement

SecureNova will govern every human and non-human identity behind its AI-enabled applications on the basis of a few core ideas: each identity gets only the access it needs, each one is unique, each one has someone answerable for it, its credentials are handled securely, it is verified on an ongoing basis, and it is removed promptly once it is no longer needed. AI agents and the services that support them are to be treated as distinct, identifiable principals. They may not take on the broad permissions of a human user, and they may not share credentials, unless a documented exception has been approved.

Any timeframes set out in this policy are internal service targets that SecureNova has proposed for itself. They should not be read as statutory or regulatory deadlines that apply everywhere.

1. Identity Lifecycle Management

1.1 Purpose and Scope

This section sets the rules for how SecureNova identities are created, approved, run, reviewed, suspended, rotated and finally retired. It covers human users as well as AI agents, OAuth machine-to-machine clients, LLM provider integrations, RAG pipeline services, MCP servers, API services and any other automated workload.

1.2 Identity types and ownership

Identity Type	What it is used for	Who is accountable
Human User	Signs in to the application using an authenticated account	The named user; the application owner manages their access
AI Agent (Orchestrator / Worker)	Carries out specific AI tasks and calls tools or APIs that have been approved for it	A named service owner together with a technical custodian
Oauth M2M Client	Gets tokens so it can call a protected API with no human involved	Application or service owner
LLM Provider integration	Proves SecureNova's identity to an external model provider	AI platform owner
RAG Pipeline service identity	Pulls in, processes and indexes knowledge sources that have been cleared for use	Owner from the data or AI platform team
MCP server identity	Verifies a tool server and the operations it is allowed to perform	Tool or service owner

API or internal service identity	Allows services to talk to one another with authentication	The engineering team that owns the service

1.3 Provisioning and approval

1. The service owner starts the process by submitting a provisioning request. It needs to give the business purpose, the system and environment involved, the data classification, the permissions required, how long the identity is expected to be needed, and the person accountable for it.

2. The application owner confirms that the access is genuinely needed. The Security Team then takes a look at any privileged identity, any external integration, and any identity able to reach sensitive data or run tools with serious consequences.

3. Every agent, service and environment gets its own identity. Identities for development, testing and production must be kept apart.

4. Give the identity no more permission than the job requires. Wherever the platform allows it, define access through specific OAuth scopes, roles, resource audiences and tool allowlists.

5. Credentials belong in an approved secrets-management tool. Secrets must never appear in prompts, retrieved documents, source code, screenshots, tickets or configuration files held under version control.

6. Before the identity is switched on, log the approval, the identity's metadata and its starting permissions in the identity inventory.

1.4 Access assignment and review

Activity	Howoften / target	Responsible role
Check privileged agent and service identities	Every month	Identity owner and Security Team
Go through every active non-human identity	Every quarter at the least	Service Owner
Re-check permissions when a role, purpose or architecture changes	Ahead of the change, or right after it	Application owner
Look at dormant identities	Every quarter at the least; disable any that are no longer needed	Identity Owner
Confirm the identity inventory is complete	Every quarter	Security Team

Each access review should establish that there is still a real business purpose, a clearly accountable owner, suitable permissions, valid credentials and the monitoring that is expected. Any identity that is unused or has no owner is to be disabled until it has been looked at.

1.5 Credential and Identity rotation

Section 2 contains the detailed rotation requirements. If a cryptographic key or credential is suspected or known to be compromised, rotate it straight away. Replacing a key also means updating the trusted public-key registrations, any services that depend on it, and the audit records. The new credential should be tested before the old one is retired, unless the old one has to be revoked at once to contain an incident.

1.6 Suspension and decommissioning

Trigger	Action required	SecureNova target
Suspected compromise	Disable or isolate the identity, revoke any exposed credentials, and block privileged actions	Start straight away; aim to have it contained within 15 minutes of confirmation
Retirement request approved	Revoke tokens, keys, client credentials, sessions, API access and tool permissions	Within 4 hours
Inventory and evidence update	Flag the identity as retired, log the revocation and keep the audit records	Within 24 hours
Legal or security hold	Keep the records that are needed and follow the authorised retention instructions	As instructed by the incident or legal owner

Decommissioning should never mean erasing the identity record. Instead, mark it as inactive or retired, so that ownership, approvals, access history and proof of revocation can still be audited.

1.7 Audit and evidence requirements

• Keep an inventory that lists both active and retired non-human identities.

• Log provisioning approvals, changes to roles and scopes, credential rotations, access reviews, and every disablement and revocation.

• Where possible, keep timestamps synchronised, and limit who can alter the audit logs.

• Logs must never contain raw passwords, API keys, private keys, access tokens or refresh tokens.

• Hold on to records for as long as the approved retention schedule and any applicable legal requirements demand.

2. Credential Governance

2.1 Credential Principle

Every credential has to be traceable to one specific identity, shielded from disclosure and misuse, restricted in both privilege and lifespan, and capable of being revoked. Where the technology allows, SecureNova will favour short lived, audience-restricted tokens and managed workload identity mechanisms rather than static secrets.

• Least privilege: a credential carries only the permissions the approved task calls for.

• Unique credentials: unrelated agents, services and environments do not reuse the same credential.

• Confidentiality: secrets are encrypted when stored and protected while they move across a network.

• Short lifetime: tokens and credentials expire or are rotated in line with the schedule that follows.

• Revocability: owners understand how to revoke or disable a credential quickly.

• Traceability: when credentials are issued, accessed or rotated, an audit trail exists that never captures the secret values.

2.2 Credential rotation schedule

Credential / mechanism	SecureNova requirement	Rotation / review trigger
LLM provider API key	Rotate at least every 90 days	Right away if exposure is suspected, usage looks unusual, or the owner changes
OAuth client secret (where applicable)	Rotate at least every 90 days	Right away if exposure is suspected; where available, managed identity or private-key options are preferred
OAuth access token	Short-lived; the project demonstration uses a 300-second lifetime where the platform supports it	Expires on its own; a new token is requested through an authorized flow
OAuth refresh token	Enable rotation where it is supported, and securely keep only the token that is currently valid	Rotate each time it is used; revoke the whole token family if replay or compromise is suspected

Ed25519 signing private key	Held in an approved key store; reviewed every quarter at the least	Rotate if compromise is suspected or the trust relationship changes; the key management owner sets the planned interval
Database or service password	Kept in a secrets manager and never hardcoded	Every 90 days at minimum wherever static passwords cannot be avoided, and at once upon compromise
Public verification key	Distributed via an authenticated, controlled registration process	Update whenever its matching private key is rotated or revoked

The 90-day ceiling is an internal SecureNova rule covering in-scope API keys and static client or service secrets. Where it is possible, use shorter lifetimes or automatic rotation. An exception can only be granted with written approval, compensating controls and a defined expiry date.

2.3 Prohibition on long-lived agent credentials

AI agents are not to be given permanent credentials, nor credentials that last longer than they need to. Whenever the platform allows it, an agent should get short-lived, scoped tokens from an approved identity provider or workload identity mechanism. A long-lived static credential is acceptable only when there is no practical alternative and the Security Team has approved a documented exception.

• Never embed API keys, client secrets, private keys or refresh tokens in system prompts, retrieved documents, source code or an agent's memory.

• Avoid broad administrative scopes whenever a narrower, task-specific scope will do.

• A human user's password must never be used as an agent's credential.

• Keep credentials for development, test and production separate.

• The service receiving a token must check its audience, issuer, expiry and scopes.

2.4 Secrets storage and handling

1. Secrets belong in an approved secrets manager, a hardware-backed key store, or a protected runtime secret facility. Local environment files are tolerated only for controlled development work, and they must never be committed to version control.

2. Only the service identities and operators who genuinely need a secret should be able to retrieve it. Use role based access, and add multifactor authentication for administrative access where it is supported.

3. Encrypt secrets while they are stored, and deliver them only over TLS-protected channels.

4. Raw secret values must never show up in terminal output, application logs, telemetry, screenshots, tickets or incident reports.

5. Demonstrations and testing should rely on synthetic credentials. If a real credential is captured by accident, redact it and then rotate it.

6. Maintain a credential inventory recording the owner, purpose, system, creation date, last rotation, next rotation due and method of revocation. The inventory itself must not hold any secret values.

2.5 Access control, monitoring and emergency rotation

Access to secrets should be logged and reviewed. Set up alerts for unusually heavy retrieval, access from an unfamiliar workload or location, use outside an approved environment, repeated authentication failures, and any attempt to use a revoked or expired credential.

If a credential is exposed, the first step is to revoke or disable it. Next, work out how it has been used since the earliest point at which it could plausibly have been exposed, and keep the related metadata and logs. Issue a replacement through the approved process, update every service that depends on it, and confirm that the old credential no longer works. A rotation only counts as finished once the old credential is revoked and the new one has been verified.

3. Incident Response Playbook

3.1 Response process and severity

This playbook is for any suspected or confirmed compromise that touches AI identities, credentials, prompts, retrieved content, model outputs or tool execution. The incident lead runs the response. The Security Team safeguards the evidence and directs containment, while system and data owners help assess the impact and carry out recovery.

Severity	Example	Initial response
Critical	Sensitive data is confirmed to have been exfiltrated, a privileged agent is actively compromised, or an exposed production credential is being misused	Escalate and contain immediately

High	A credible suspicion of agent compromise, a prompt injection that tries to extract sensitive information, or privileged tool use that looks suspicious	Triage quickly; aim to contain within 15 minutes of confirmation
Medium	An attack attempt that was blocked, a one off policy violation, or suspicious activity with no sign of compromise	Triage within the same working day
Low	An informational event, or a false positive with no meaningful security impact	Log it and review it as part of normal operations

These figures are proposed operational targets. The incident commander can respond faster than this, depending on how sensitive the data is, how critical the service is, how far the access reaches and whether there is evidence of active exploitation.

Standard Response Steps

1. Detect and record: open an incident record containing a unique ID, the time of detection, the reporter, the affected system and the first indicators.
2. Triage: establish whether the event is suspected or confirmed, work out which identities and data are affected, and judge whether the activity is still going on.
3. Contain: put a stop to the harmful activity, disable the affected identities or credentials, limit tool and API access, and isolate affected components where that makes sense.
4. Preserve evidence: keep the relevant logs, event metadata, configuration snapshots, source documents, message IDs and timestamps, and maintain a chain of custody where one is required.
5. Notify: tell the incident lead, the security owner, the service owner and, depending on severity and any obligations that apply, the privacy and legal stakeholders.
6. Eradicate and recover: remove malicious content and any access paths, rotate credentials, bring back known good configurations, and confirm the controls work before the service is restored.
7. Close and improve: write up the root cause, the impact, the actions taken, the decisions made, the lessons learned and who owns each remediation.

Gathering evidence should never hold up urgent containment. Exposed secrets must not be copied into incident records; note identifiers, fingerprints, locations and timestamps in their place.

3.2 Scenario-A – Leaked LLM API Key

Scenario: An LLM provider API key becomes exposed, whether through a public repository, a terminal capture, an application log, a ticket, a developer workstation or some other unauthorized channel.

Response area	Actions required
Detection Indicators	An alert from secret scanning or from the provider; model usage, request volume or cost that nobody expected; the key turning up in a public repository or log; access coming from a workload that isn't recognized.
Containment	Revoke or disable the exposed key at the provider and block any further use of it. Suspend the affected integration if that proves necessary. Deleting the visible copy is not enough by itself.
Evidence handling	Note the key identifier or fingerprint if there is one, the repository and commit references, the exposure window, timestamps, provider usage records, source IP or workload metadata, and the systems affected. The raw key itself must not be kept in the report.
Notification	Inform the Security Team, the LLM integration owner, the application owner and, where appropriate, the provider's contact. If personal or regulated data may have been exposed, escalate to the privacy and legal stakeholders.
Recovery	Issue a new key and store it in the secrets manager. Update the integration and test the service, then check that the old key is rejected. Go over usage and billing, and scan repositories and logs for any other exposure.

3.3 Scenario-B – Compromised agent identity

Scenario: An attacker steals or impersonates an AI agent's identity, replays its token or signing key, or sends a forged message asking for privileged actions.

Response Area	Actions Required
Detection indicators	An Ed25519 signature that fails validation; a public key that is unknown or has been revoked; an agent message that looks out of character; a request for a scope or role nobody expected; repeated token failures; abnormal tool calls; activity from a workload that has not been approved.
Containment	Disable or isolate the affected agent. Revoke its access and refresh tokens, and revoke or quarantine the compromised signing key. Block privileged operations and narrow the related API scopes and tool permissions.

Evidence Handling	Keep the message envelopes, sender and recipient IDs, signature result, key ID, token metadata (but not the token value itself), audit logs, request IDs, timestamps and the resources affected.
Notification	Let the Security Team, the agent owner, the identity provider administrator, the API or service owner and the incident lead know. If sensitive resources may have been accessed, the data owners should be told as well
Recovery	Re-create the identity through the approved provisioning process. Generate a fresh key pair in a protected environment and register the new public key over an authenticated channel. Restore the approved scopes, test verification, and keep monitoring.

A failed signature check is a warning sign, but on its own it does not prove that a compromise succeeded. Cross check it against identity, token, workload and access logs.

3.4 Scenario-C – Prompt Injection leading to data exfiltration

Scenario: Malicious instructions hidden in a user prompt, a retrieved document, external content or a tool result sway an AI agent, and the agent then reveals confidential information or passes it on through an output or a tool.

Response Area	Actions Required
Detection Indicators	A sensitive-output redaction or DLP alert; response content that seems unusual; a tool invoked unexpectedly; abnormal data retrieval; access that falls outside the task; a suspicious outbound transfer; a retrieved document that has been poisoned or altered.
Containment	Halt the affected agent workflow. Block the tool or outbound data path involved, limit access to the affected records, and quarantine the suspect document or source. Disable unsafe tool permissions until they have been reviewed.
Evidence handling	Keep the original input, the retrieved document and its source metadata, prompt and context references, the model output, the tool-call request and result, access logs, data identifiers and timestamps. Whatever personal or confidential data is collected must itself be protected.
Notification	Notify the incident lead, the Security Team, the AI service owner and the data owner. Bring in the privacy, legal and customer communications teams where an actual disclosure or transfer could create a duty to notify.
Recovery	Remove or quarantine the malicious content and check where its source came from. Rotate any credentials that were exposed, review permissions and egress controls, and tighten the guardrails around inputs, outputs and tools. Re-test the attack path, and restore the service only once approval is given.

Not every prompt-injection attempt amounts to a data-exfiltration incident. This scenario applies when the evidence shows that sensitive information was really disclosed or transferred, or when the incident commander decides that full incident handling is warranted.

3.5 Post-incident review and lessons learned

• Once containment and recovery are done, carry out a post-incident review that is in proportion to the severity of the incident.

• Write down the timeline, the root cause, the identities and systems affected, the impact on data, the containment decisions taken and how recovery was validated.

• Pinpoint where controls failed, such as missing ownership, excessive privileges, weak credential handling, untrusted retrieval sources or inadequate monitoring.

• Turn the findings into remediation actions, each with a named owner, a due date and a measurable definition of done.

• If the findings alter the assessed risk, update the threat model, risk register, policy, compliance mapping and test cases accordingly.

• Confirm that corrective actions work by running a controlled retest.